

Generics in Java

Lecture 02 — Advanced Programming

Ayoub FRIHAOUI

Rezak AZIZ rezak.aziz@lecnam.net

Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.

Source: <https://rezakaziz.github.io/PROG-AVANC/>

The origin of the problem

```
public class StringList {
    private String[] l = new String[100];
    private int size;
    public void add(String s) { l[size++] = s; }
    public String get(int i) { return l[i]; }
}

public class IntegerList {
    private Integer[] l = new Integer[100];
    private int size;
    public void add(Integer s) { l[size++] = s; }
    public Integer get(int i) { return l[i]; }
}
```

The origin of the problem

```
public class StringList {  
    private String[] l = new String[100];  
    private int size;  
    public void add(String s) { l[size++] = s; }  
    public String get(int i) { return l[i]; }  
}  
  
public class IntegerList {  
    private Integer[] l = new Integer[100];  
    private int size;  
    public void add(Integer s) { l[size++] = s; }  
    public Integer get(int i) { return l[i]; }  
}
```

Observation?

- Same logic, only the type changes
String $\rightarrow$ Integer
- Code duplication

How can we fix this problem?

A first solution: the Object class

```
public class ObjectList {  
    private Object[] l = new Object[100];  
    private int size;  
    public void add(Object s) {  
        l[size++] = s;  
    }  
    public Object get(int i) {  
        return l[i];  
    }  
}  
  
ObjectList texts = new ObjectList();  
texts.add("Hello");  
String s = (String) texts.get(0); // cast!
```

Advantage

- One single code for all types;

A first solution: the Object class

```
public class ObjectList {  
    private Object[] l = new Object[100];  
    private int size;  
    public void add(Object s) {  
        l[size++] = s;  
    }  
    public Object get(int i) {  
        return l[i];  
    }  
}  
  
ObjectList texts = new ObjectList();  
texts.add("Hello");  
String s = (String) texts.get(0); // cast!
```

Advantage

- One single code for all types;

Limits?

- Loss of type information:
everything becomes an Object;
- Mandatory cast at each retrieval;

Parametric polymorphism

```
public class MyList<T> {  
    private T[] l = new T[100]; // error  
    private int size;  
  
    public void add(T s) {  
        l[size++] = s;  
    }  
  
    public T get(int i) {  
        return l[i];  
    }  
}  
  
MyList<String> texts = new MyList<>();  
texts.add("Hello");  
String s = texts.get(0); // no cast
```

Key idea

Replace the concrete type by a generic parameter `<T>`:

- T is a formal type chosen at instantiation;
- the compiler checks type consistency;
- no more explicit casts.

Parametric polymorphism

```
public class MyList<T> {  
    private T[] l = new T[100]; // error  
    private int size;  
  
    public void add(T s) {  
        l[size++] = s;  
    }  
  
    public T get(int i) {  
        return l[i];  
    }  
}  
  
MyList<String> texts = new MyList<>();  
texts.add("Hello");  
String s = texts.get(0); // no cast
```

Key idea

Replace the concrete type by a generic parameter `<T>`:

- T is a formal type chosen at instantiation;
- the compiler checks type consistency;
- no more explicit casts.

Warning

THIS CODE IS WRONG IN JAVA!!

A small technical problem

```
public class MyList<T> {  
    private Object[] l = new Object[100];  
    private int size;  
  
    public void add(T s) {  
        l[size++] = s;  
    }  
  
    public T get(int i) {  
        return (T) l[i]; // cast needed  
    }  
}  
  
MyList<String> texts = new MyList<>();  
texts.add("Hello");  
String s = texts.get(0); // typing at  
                        // compilation
```

Warning

Java arrays and parametric polymorphism do not get along well.

A small technical problem

```
public class MyList<T> {  
    private Object[] l = new Object[100];  
    private int size;  
  
    public void add(T s) {  
        l[size++] = s;  
    }  
  
    public T get(int i) {  
        return (T) l[i]; // cast needed  
    }  
}  
  
MyList<String> texts = new MyList<>();  
texts.add("Hello");  
String s = texts.get(0); // typing at  
                        // compilation
```

Warning

Java arrays and parametric polymorphism do not get along well.

Solution?

Impossible to create an array of a generic type directly:

`new T[100]` → forbidden.

Workaround with an `Object[]` and a controlled cast.

With more than one type parameter

```
public class Pair<A, B> {  
    private A first;  
    private B second;  
  
    public Pair(A first, B second) {  
        this.first = first;  
        this.second = second;  
    }  
  
    public A getFirst() { return first; }  
    public B getSecond() { return second; }  
}  
  
Pair<String, Integer> c =  
    new Pair<>("house", 4);
```

Principle

A class can be parameterised by several types:

<A, B, C, ...>

Typical use

- Key/value pairs: Map<K,V>;

Upper bound

```
public class SortedList<T extends Comparable<T>> {  
    private List<T> elements = new ArrayList<>();  
  
    public void add(T element) {  
        elements.add(element);  
        Collections.sort(elements);  
    }  
  
    public T get(int i) {  
        return elements.get(i);  
    }  
}  
  
SortedList<String> names = new SortedList<>();  
names.add("Alice");  
names.add("Bob");
```

Principle

An **upper bound** limits the accepted types. For example, T must implement Comparable<T>.

Several bounds can be combined:

`<T extends X & Y & Z>`

Generic methods

```
public class Utils {  
  
    public static <T extends Comparable<T>>  
    T max(T a, T b) {  
        return (a.compareTo(b) > 0) ? a : b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(max(4, 7));  
        System.out.println(max("abc", "def"));  
    }  
}
```

Principle

A method can be **generic**, even if the class is not. The method declaration is preceded by the generic types.

To remember

- Generic methods define their own type <T>;
- They are often static;
- Strong typing is kept, with no cast.

Type erasure

```
public class Sums {  
    public static int sum(List<Integer> l)  
    { ... }  
    public static double sum(List<Double> l)  
    { ... }  
}
```

```
// At runtime:  
List<String> a = new ArrayList<>();  
List<Integer> b = new ArrayList<>();  
  
System.out.println(  
    a.getClass() == b.getClass()  
); // true
```

Principle

Generics exist only at **compilation**. The parameterised type is **erased** in the bytecode.

Consequences

- No overloading on generic types.
- No access to the generic type at runtime (introspection);
- All `List<T>` become `List`.

Liskov Substitution Principle

Liskov substitution principle

Definition

Barbara Liskov (Turing Award 2008):

If B is a subclass of A, then any object of type B can be used wherever an object of type A is expected.

Liskov substitution principle

Definition

Barbara Liskov (Turing Award 2008):

If B is a subclass of A, then any object of type B can be used wherever an object of type A is expected.

Formal statement

For any predicate P verified by the objects of class A, $P(b)$ must stay true for any object b of class B:

$$(B \text{ inherits } A) \implies (P(A) \implies P(B))$$

Liskov substitution principle

Definition

Barbara Liskov (Turing Award 2008):

If B is a subclass of A, then any object of type B can be used wherever an object of type A is expected.

Formal statement

For any predicate P verified by the objects of class A, $P(b)$ must stay true for any object b of class B:

$$(B \text{ inherits } A) \implies (P(A) \implies P(B))$$

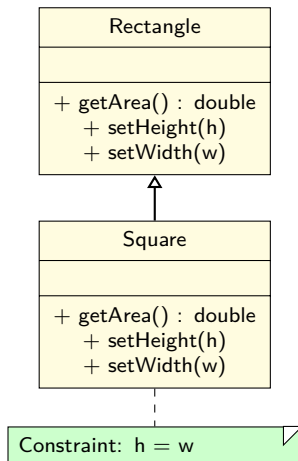
In practice

- The child class must not restrict the contract of the parent class;
- Inherited methods must stay consistent.

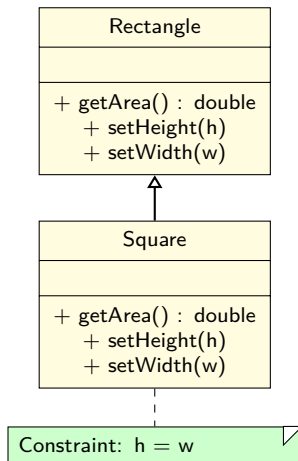
Classical example

Square vs Rectangle: inheriting does not always mean respecting Liskov.

Problem: Square vs Rectangle



Problem: Square vs Rectangle



Observation

A square logically inherits from a rectangle, but imposes an additional constraint (restricts the contract):

$$\text{height} = \text{width}$$

```
void testRectangle(Rectangle rect) {
    rect.setWidth(5);
    rect.setHeight(10);
    int area = rect.getArea();
    assert area == 50;
}
```

Liskov: pre-conditions and post-conditions

If B inherits from A, the methods of B must respect:

- A subtype B must **not strengthen** the preconditions of its parent A.
If the parent asks for an Animal, the child cannot strengthen it by asking for a Cat.

Liskov: pre-conditions and post-conditions

If B inherits from A, the methods of B must respect:

- A subtype B must **not strengthen** the preconditions of its parent A.

If the parent asks for an Animal, the child cannot strengthen it by asking for a Cat.

- A subtype B must **not weaken** the postconditions of its parent A.

If the parent provides an Animal, the child may strengthen it by providing a Cat.

Liskov: pre-conditions and post-conditions

If B inherits from A, the methods of B must respect:

- A subtype B must **not strengthen** the preconditions of its parent A.
If the parent asks for an Animal, the child cannot strengthen it by asking for a Cat.
- A subtype B must **not weaken** the postconditions of its parent A.
If the parent provides an Animal, the child may strengthen it by providing a Cat.
- **Invariants** must be preserved.

Example: Square and Rectangle?

- The invariant of `Rectangle.setWidth()` is “width and height must always stay strictly positive”;
- The invariant of `Square.setWidth()` is “width and height must be positive and equal”.

Liskov: pre-conditions and post-conditions

If B inherits from A, the methods of B must respect:

- A subtype B must **not strengthen** the preconditions of its parent A.
If the parent asks for an Animal, the child cannot strengthen it by asking for a Cat.
- A subtype B must **not weaken** the postconditions of its parent A.
If the parent provides an Animal, the child may strengthen it by providing a Cat.
- **Invariants** must be preserved.

Example: Square and Rectangle?

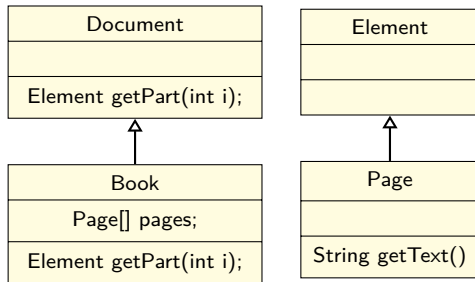
- The invariant of `Rectangle.setWidth()` is “width and height must always stay strictly positive”;
- The invariant of `Square.setWidth()` is “width and height must be positive and equal”.

LSP violation

Square does not respect the contract of Rectangle: a square cannot replace a rectangle without surprise.

Covariance and Contravariance

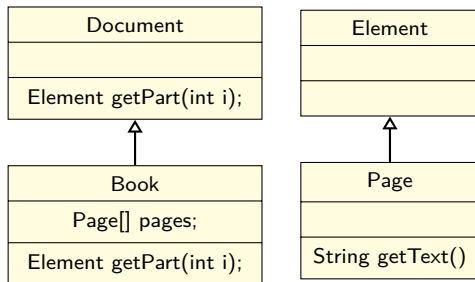
Example: which return type is more appropriate?



Redefining the method `getPart` of **Document** in **Book** is correct. **But ...**

```
Element getPart(int i) { return pages[i]; }
```

Example: which return type is more appropriate?



```
Element getPart(int i) { return pages[i]; }
```

Redefining the method `getPart` of `Document` in `Book` is correct. **But ...**

Drawback?

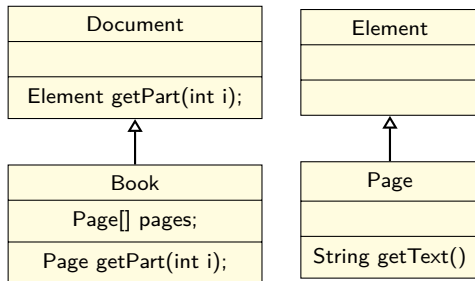
If I write:

```
Element p = book.getPart(1);
```

I cannot write:

```
String txt = p.getText();
```

Example: which return type?



```
Page getPart(int i) { return pages[i]; }
```

Redefining `getPart` is correct. **But ...** it is more logical to change the return type to `Page`.

Advantage

I can write:

```
Page p = book.getPart(1);
String txt = p.getText();
```

And it respects Liskov: everything I can call on a `Document` is doable with a `Book`.

Covariance

Definition

We speak of **covariance** when a more specific type can be used in place of a more general type.

`B extends A` $\implies$ B can replace A

If:

`Page extends Element`

and

`Book extends Document`

Then the return types of `getPart()`
in `Book` and `Document` are covariant.

Covariance

Definition

We speak of **covariance** when a more specific type can be used in place of a more general type.

$B \text{ extends } A \implies B \text{ can replace } A$

If:

Page extends Element

and

Book extends Document

Then the return types of `getPart()`
in Book and Document are covariant.

Warning

To respect LSP, covariance applies:

- to **return types** (since Java 1.5);
- **not** to method arguments ($\rightarrow$ contravariance).

Summary

Covariance = “may return a more specific type”. It respects the Liskov substitution principle.

Contravariance

According to Liskov

- If B extends A
- and A has $f(A1\ x)$
- then B may have $f(B1\ x)$
- provided B1 is *more general* than A1 (see the LSP preconditions)
- So A1 extends B1

Contravariance

Contravariance = accepting a more general type where a more specific type is expected.

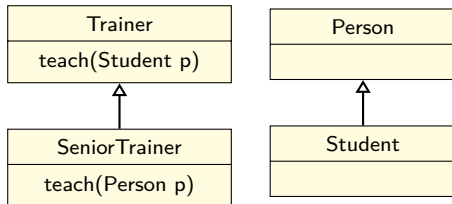
Contravariance

According to Liskov

- If B extends A
- and A has $f(A1\ x)$
- then B may have $f(B1\ x)$
- provided B1 is *more general* than A1 (see the LSP preconditions)
- So A1 extends B1

Contravariance

Contravariance = accepting a more general type where a more specific type is expected.



Warning

This does not work in Java.

Problem in Java

```
class Person { }  
class Student extends Person { }  
  
List<Person> lp = new ArrayList<>();  
List<Student> ls = new ArrayList<>();  
  
lp = ls; // Possible or not?
```

Problem in Java

```
class Person { }  
class Student extends Person { }  
  
List<Person> lp = new ArrayList<>();  
List<Student> ls = new ArrayList<>();  
  
lp = ls; // Possible or not?
```

Impossible

```
// If it were allowed:  
lp.add(new Person()); // OK for the compiler  
Student e = ls.get(0); // ClassCastException!
```

Problem in Java

```
class Person { }  
class Student extends Person { }  
  
List<Person> lp = new ArrayList<>();  
List<Student> ls = new ArrayList<>();  
  
lp = ls; // Possible or not?
```

Impossible

```
// If it were allowed:  
lp.add(new Person()); // OK for the compiler  
Student e = ls.get(0); // ClassCastException!
```

Consequence

Java refuses implicit contravariance to avoid any inconsistency in write operations. This guarantees **type safety**:

- no invalid objects added to a collection;
- no type error at runtime;
- no `ClassCastException`.

Moreover

Java generics are **invariant**.

Solution: bounded wildcards

Wildcards: ? extends and ? super

? extends T — covariant (reading)

```
List<? extends Person> list;
```

```
Person p = list.get(0);
```

```
// reading allowed
```

```
list.add(new Student());
```

```
// writing forbidden
```

→ “Producer” of T (we read T or subtypes of T).

Wildcards: ? extends and ? super

? extends T — covariant (reading)

```
List<? extends Person> list;
```

```
Person p = list.get(0);
```

```
// reading allowed
```

```
list.add(new Student());
```

```
// writing forbidden
```

→ “Producer” of T (we read T or subtypes of T).

? super T — contravariant (writing)

```
List<? super Student> list;
```

```
list.add(new Student());
```

```
// writing allowed
```

```
Student e = list.get(0);
```

```
// reading forbidden
```

→ we can pass Students or supertypes of Student (the list consumes Student).

Wildcards: ? extends and ? super

? extends T — covariant (reading)

```
List<? extends Person> list;
```

```
Person p = list.get(0);  
// reading allowed  
list.add(new Student());  
// writing forbidden
```

→ “Producer” of T (we read T or subtypes of T).

? super T — contravariant (writing)

```
List<? super Student> list;
```

```
list.add(new Student());  
// writing allowed  
Student e = list.get(0);  
// reading forbidden
```

→ we can pass Students or supertypes of Student (the list consumes Student).

Goal

Preserve type safety while keeping the flexibility of generics.

Summary: covariance, contravariance and invariance

The three forms of variance

- **Covariance**: accepts a more specific type. Example: `? extends T`
- **Contravariance**: accepts a more general type. Example: `? super T`
- **Invariance**: accepts only an exact type. Example: `List<T>`

Reminder of the PECS rule

Producer Extends, Consumer Super

- `? extends T` → reading (producer)
- `? super T` → writing (consumer)

Conclusion

Java favours **invariance** by default to guarantee type safety.